`ConfigureProductionServices`, and ASP.NET Core invokes the correct method based on the environment. You can even create a whole `StartupProduction` class if you wish! For more details on these Startup conventions, see the documentation at http://mng.bz/Oxxw.

The `Startup` class isn't the only place where ASP.NET Core relies on opaque conventions. You may remember in section 30.2 I mentioned that Program.cs deliberately extracts the building of the `IHostBuilder` to a method called `CreateHostBuilder`. The name of this method was historically important. Tooling such as the EF Core tools hooked into it so that they could load your application configuration and services when running migrations and other functionality. In earlier versions of ASP.NET Core, renaming this method would break all your tooling!

NOTE As of .NET 6, you don't have to create a `CreateHostBuilder` method; you can create your whole app inside your `Main` function (or using top-level statements), and the EF Core tools will work without error. This was fixed partly to add support for `WebApplication`. If you're interested in the mechanics of how it was fixed, see my blog at http://mng.bz/Y11z.

Once you're experienced with ASP.NET Core, most of these gripes become relatively minor. You quickly get used to the standard patterns and avoid the pitfalls. But for new users of ASP.NET Core, Microsoft wanted a smoother experience, closer to the experience you get in many other languages.

The minimal hosting APIs provided by `WebApplicationBuilder` and `WebApplication` largely address these concerns. Configuration happens all in one file using an imperative style, with far fewer lambda-based configuration methods or implicit convention-based setup. All the